FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento: Eléctrica y Electrónica
	Carrera: Electrónica y Automatización Asignatura: Fundamentos de la Programación NRC:28561
	Apellidos y nombres del estudiante: Figueroa Cerón Sofía Vanessa

Ejercicio 11

Ejercicio:

11. Leer el precio de un producto y la cantidad comprada

Resultado esperado	Leer el precio de un producto y la cantidad comprada, validar que ambos valores sean mayores que cero y calcular el total a pagar.

Contexto del ejercicio

Se presenta un programa que solicita el precio de un producto y la cantidad comprada por teclado.La condición principal es verificar que el precio y la cantidad sean mayores que cero. Si los datos son válidos, se calcula el total multiplicando el precio por la cantidad.

Regla de cálculo:

total <- precio * cantidad

1. Seudocódigo para PSeInt

```
Inicio

   Definir precio, cantidad, total Como Real


   Escribir "Ingrese el precio del producto:"

   Leer precio


   Escribir "Ingrese la cantidad comprada:"

   Leer cantidad


   Si precio > 0 Y cantidad > 0 Entonces

      total <- precio * cantidad
```

```
        Escribir "El total a pagar es: ", total

    Sino

        Escribir "Datos inválidos"

    FinSi

Fin
```

2. Prueba de escritorio en PSeInt

Paso	precio	cantidad	total	Salida	Descripción
1					Se declaran las variables precio, cantidad y total.
2	2.50				Se ingresa el precio del producto.
3	2.50	4			Se ingresa la cantidad comprada.
4	2.50	4	10		Se calcula total <- precio * cantidad.
5	2.50	4	10	El total a pagar es:10.	Se muestra el resultado en pantalla.

Salida esperada al ingresar:

Precio del producto: 2.50
Cantidad comprada: 4

El total a pagar es: 10

3. Guía para pasar de PSeInt a Code::Blocks con sintaxis validada

Ahora sí: pasamos la lógica a lenguaje C. La clave es traducir cada instrucción de PSeInt a su equivalente en C, sin perder el orden de los ciclos.

Pseint	C / Code::Blocks	Para Recordar
Definir precio, cantidad, total Como Real	float precio, cantidad, total;	Las variables reales en C se declaran con float.
Escribir "Ingrese el precio del producto:"	printf("Ingrese el precio del producto: ");	printf muestra mensajes en pantalla.
Leer precio	scanf("%f", &precio);	%f permite leer números reales.
Escribir "Ingrese la cantidad comprada:"	printf("Ingrese la cantidad comprada: ");	Se solicita la cantidad del producto.
Leer cantidad	scanf("%f", &cantidad);	El & guarda el valor dentro de la variable.
Si precio > 0 Y cantidad > 0 Entonces	if (precio > 0 && cantidad > 0) {	En C, el operador lógico **Y** se escribe &&.
total <- precio * cantidad	total = precio * cantidad;	En C se usa = para asignar un valor.
Escribir "El total a pagar es: ", total	printf("El total a pagar es: %.2f\n", total);	%.2f muestra el resultado con dos decimales.
Sino	else {	Se ejecuta cuando la condición no se cumple.
Escribir "Datos inválidos"	printf("Datos invalidos\n");	Se muestra si el precio o la cantidad no son válidos.
FinSi	}	Las llaves cierran la condición en C.

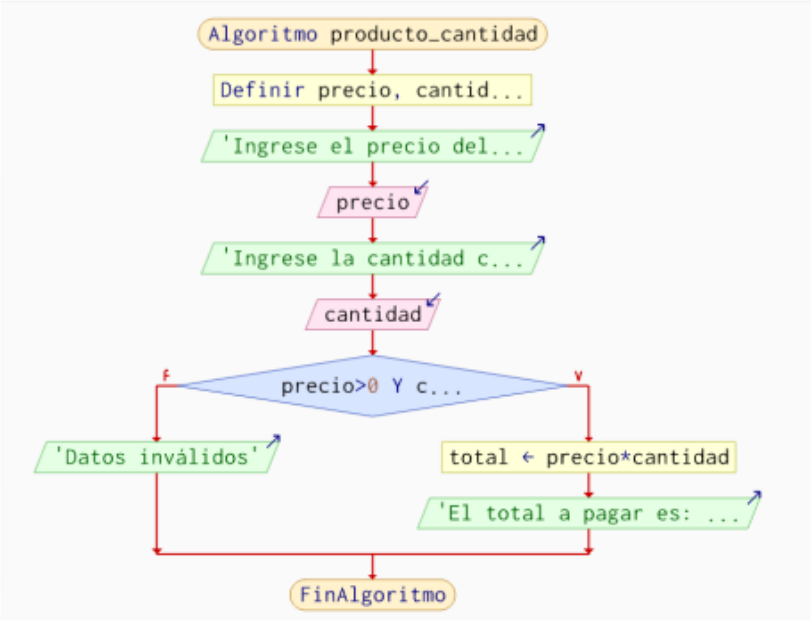
Código validado en C para Code::Blocks

```
#include <stdio.h>


int main() {

    float precio, cantidad, total;
```

```
    printf("Ingrese el precio del producto: ");

    scanf("%f", &precio);


    printf("Ingrese la cantidad comprada: ");

    scanf("%f", &cantidad);


    if (precio > 0 && cantidad > 0) {

        total = precio * cantidad;

        printf("El total a pagar es: %.2f\n", total);

    } else {

        printf("Datos invalidos\n");

    }


    return 0;

}
```

Diagrama De Flujo

Ejercicio 11



Conclusiones

El programa permite calcular correctamente el total a pagar de una compra, utilizando el precio del producto y la cantidad comprada. Además, valida que ambos datos sean mayores que cero antes de realizar la operación.

Recomendaciones

Se recomienda verificar que el precio y la cantidad ingresados sean valores positivos para evitar resultados incorrectos. También es importante usar variables reales, ya que el precio puede contener decimales.

Pasos sugeridos en Code::Blocks

1. Crear un nuevo archivo con extensión .c o abrir el archivo del ejercicio.
2. Copiar el código validado en C.
3. Compilar con Build o F9
4. Ejecutar con Run o con Build and Run.F9
5. Comparar la salida con la prueba de escritorio.

4. Rúbrica de evaluación sobre 20 puntos

Criterio	Excelente	Bueno	En proceso	Puntaje
Comprensión del problema	Identifica claramente que el triángulo debe ser hueco, con borde izquierdo, diagonal y base.	Reconoce la forma del triángulo, aunque explica parcialmente la condición de impresión.	Confunde triángulo lleno con triángulo hueco o no distingue los bordes.	4 pts
Seudocódigo en PSeInt	Usa correctamente variables, Repetir, Para, Si/Sino y Escribir Sin Saltar.	Presenta la estructura general, con pequeños errores de sintaxis o ubicación.	El seudocódigo está incompleto o no representa la lógica del ejercicio.	4 pts
Prueba de escritorio	Comprueba el algoritmo fila por fila y explica la salida para n = 10.	Realiza una prueba parcial con algunas filas o poca explicación.	No evidencia seguimiento de variables i y j o la salida no coincide.	4 pts
Traducción a C en Code::Blocks	El código compila, ejecuta y mantiene la lógica validada del triángulo hueco.	El código tiene errores menores corregibles, pero la lógica principal es adecuada.	El código no compila o no aplica correctamente FOR, if y operadores lógicos.	5 pts
Presentación y orden	Entrega el trabajo limpio, ordenado y fácil de leer, con comentarios útiles.	El trabajo es comprensible, aunque puede mejorar en orden o comentarios.	El trabajo es difícil de seguir o no evidencia organización.	3 pts